KI in TYPO3

Probleme, Fragen und Anforderungen

Erfahrungen mit nr-llm und darauf aufbauenden Anwendungen

Sebastian Mendel · Netresearch



Was euch heute erwartet

01 Möglichkeiten: fünf Anwendungen erleben

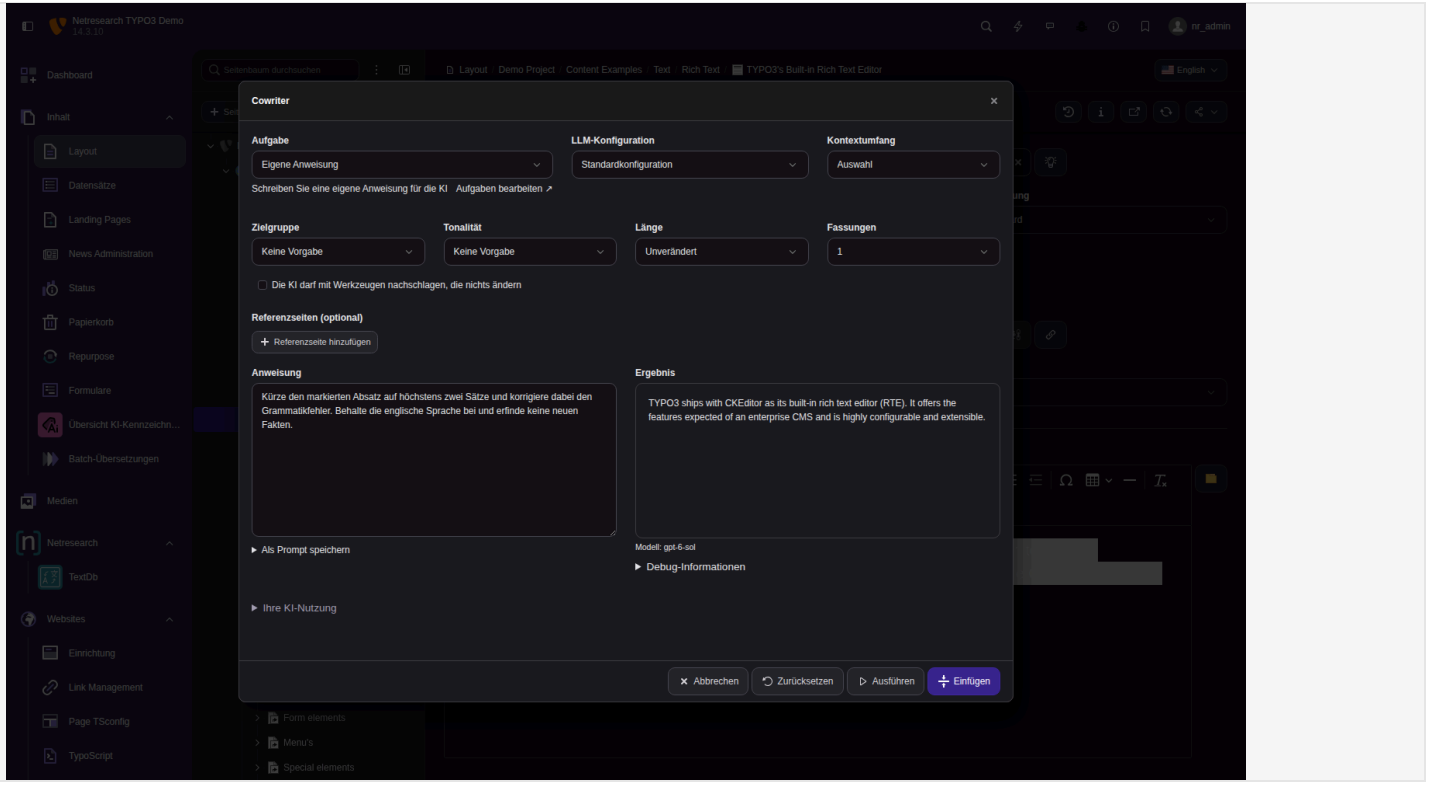
02 Fragen: Erfahrungen und offene Punkte sammeln

03 Antworten und Grenzen: am Beispiel nr-llm

04 Agentenwege: intern und extern vergleichen

05 Einordnung: andere Lösungen und Austausch

Schreibunterstützung im Editor



Cowriter-Vorschau · Aufnahme vom 2.11.2026

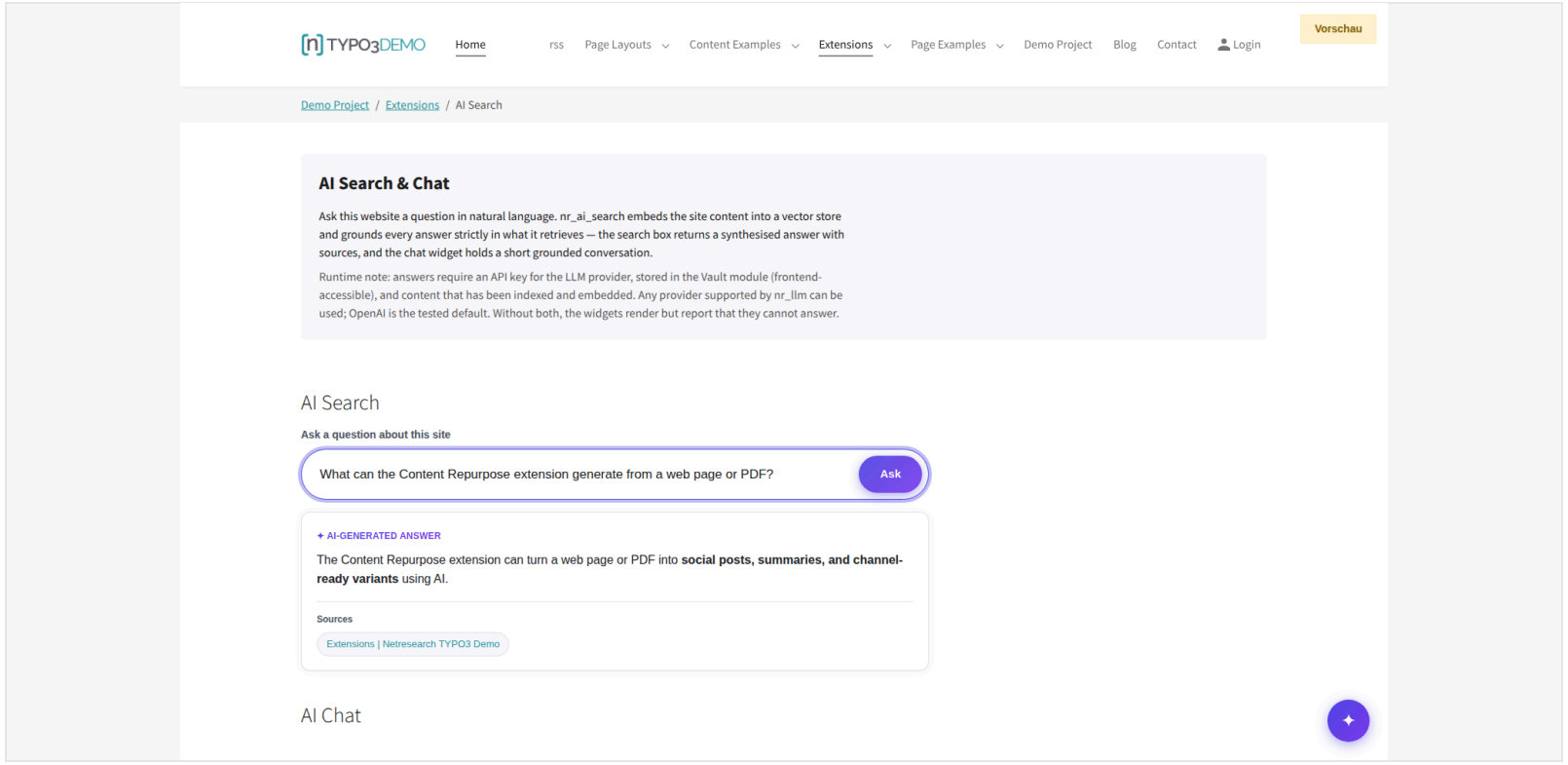
01 Ausgewählten Text überarbeiten

02 Vorschlag prüfen und übernehmen

03 Die Redaktion entscheidet über die Übernahme

Cowriter im Backend

Antworten aus Website-Inhalten



Antwort mit Quelltreffern · Aufnahme vom 2.11.2026

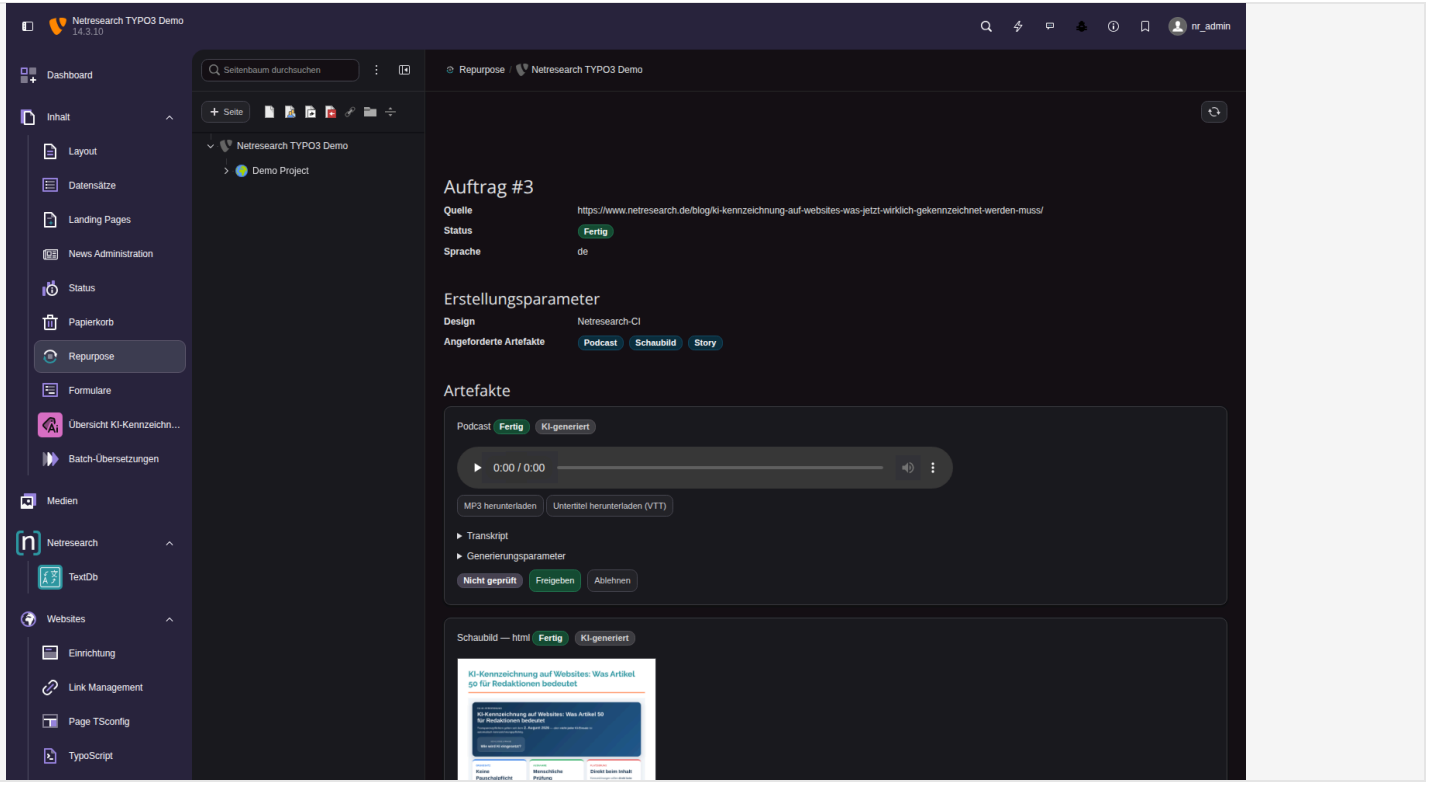
01 Antwort mit Quelle nachvollziehen

02 Quelltreffer öffnen

03 Zeigen, wenn die Quellen keine Antwort liefern

AI Search · Vorschau

Ein Inhalt, mehrere Formate



Vorbereiteter Auftrag 3 · Aufnahme vom 2.11.2026

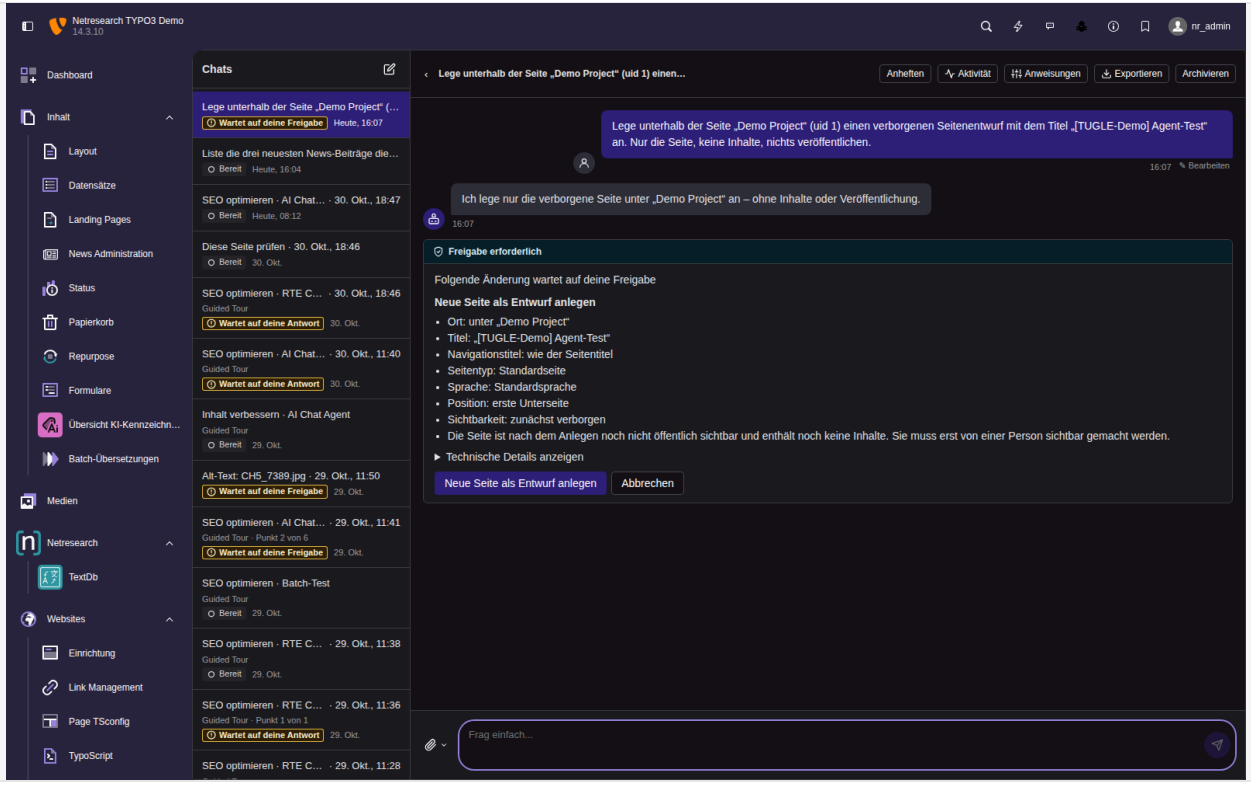
01 Ausgangsquelle bleibt nachvollziehbar

02 Schaubild und Podcast als Ergebnisse

03 Jedes Artefakt einzeln prüfen

Repurpose öffnen

Der Agent schlägt vor, ihr gebt frei



Konkrete Schreibfreigabe · Aufnahme vom 2.11.2026

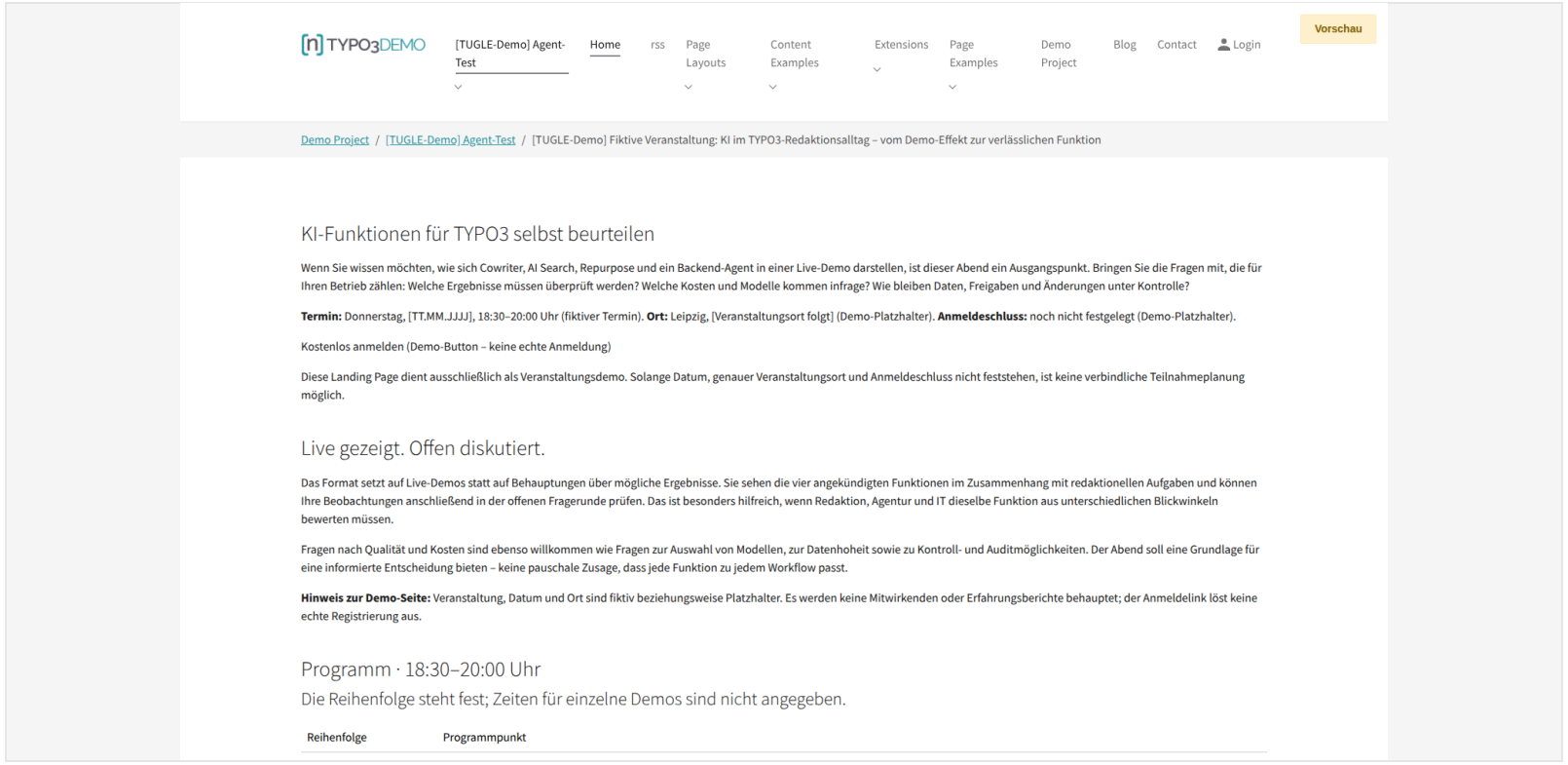
01 Daten mit Werkzeugen lesen

02 Geplante Änderung prüfen

03 Freigeben und Ergebnis kontrollieren

KI-Chat öffnen

Vom Briefing zum Seitenentwurf



Oberer Seitenbereich des Demo-Entwurfs · 2.11.2026

01 Briefing und Struktur vorgeben

02 Generierte Seite als Entwurf prüfen

03 Auch Metadaten brauchen Kontrolle

Entwurf in Vorschau

02

Fragen zum KI-Einsatz

Welche Fragen werfen diese Beispiele auf?

01 Was möchtet ihr vor dem Einsatz klären?

02 Welche Fragen kennt ihr aus euren Projekten?

03 Was ist beim Zuschauen neu aufgekommen?

Vier Fragenfelder beim KI-Einsatz

Qualität

Wie prüfen wir Ergebnisse und erkennen Fehler?

Betrieb

Wie wählen wir Modelle und begrenzen Kosten und Ausfälle?

Daten

Welche Inhalte gehen an welchen Anbieter?

Verantwortung

Wer darf welche Änderungen ausführen – und wer gibt sie frei?

03

Antworten und Grenzen

Fünf Anwendungen, eine KI-Anbindung

Anwendungen	Cowriter · AI Search · Repurpose · Landingpage · Backend-Agent
↓	
nr-llm	Dienste · Konfigurationen · Werkzeuge · Nutzungskontrolle
↓	
Provider und Modelle	Anbindung und Fähigkeiten passend zur jeweiligen Aufgabe

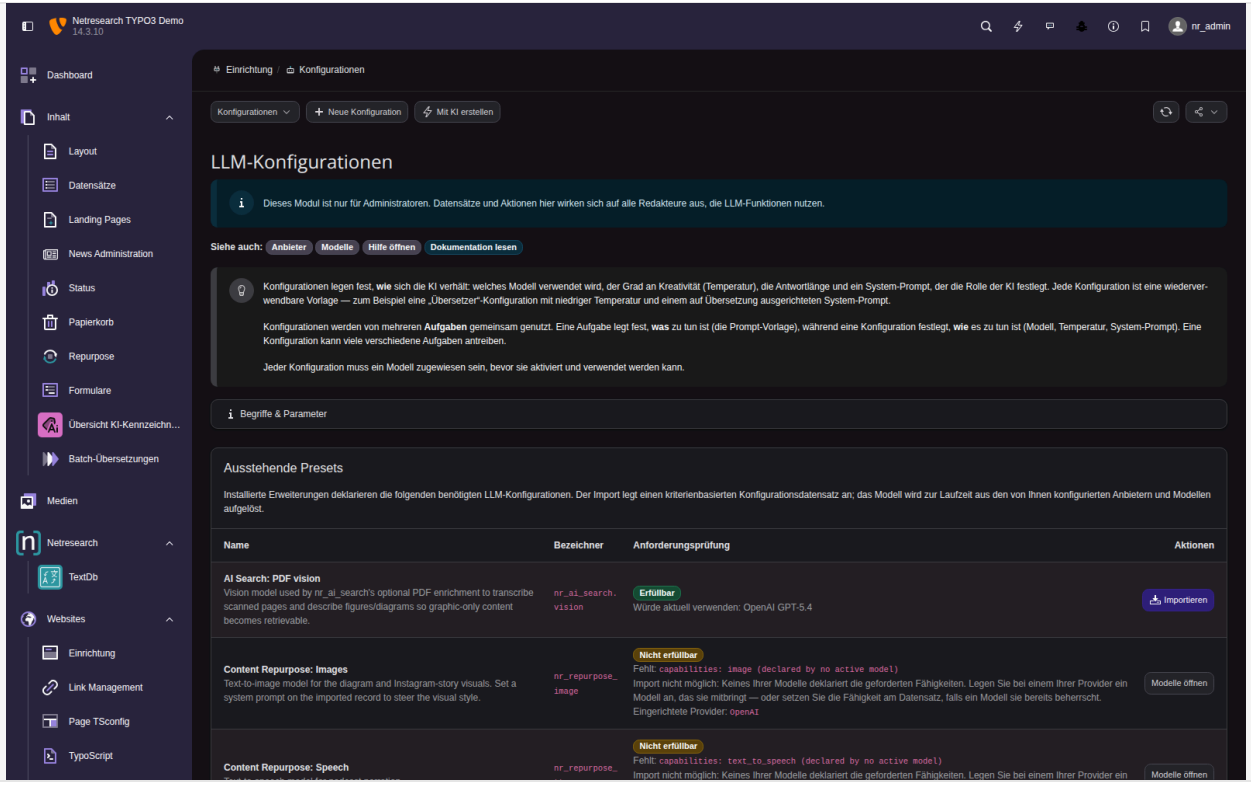
Modularer KI-Stack im Überblick

nr-llm: Projekt und Dokumentation

Anbieter, Modell und Aufgabe trennen

Provider	Verbindung, Endpunkt und Zugang zum gewählten Dienst
Modell	Fähigkeiten, Kontextfenster und Preise des gewählten Modells
Konfiguration	Systemprompt und Parameter für eine wiederkehrende Aufgabe

Wiederkehrende Aufgaben, gemeinsame Regeln



Konfigurationsübersicht · Aufnahme vom 2.11.2026

01 Systemprompt und Parameter pro Zweck

02 Tasks verbinden Eingabe und Konfiguration

03 Snippets: Textbausteine für Tonalität oder Zielgruppe

Konfigurationen öffnen

Die Anwendung liefert den Kontext

Cowriter

Der ausgewählte Absatz aus dem Editor

AI Search

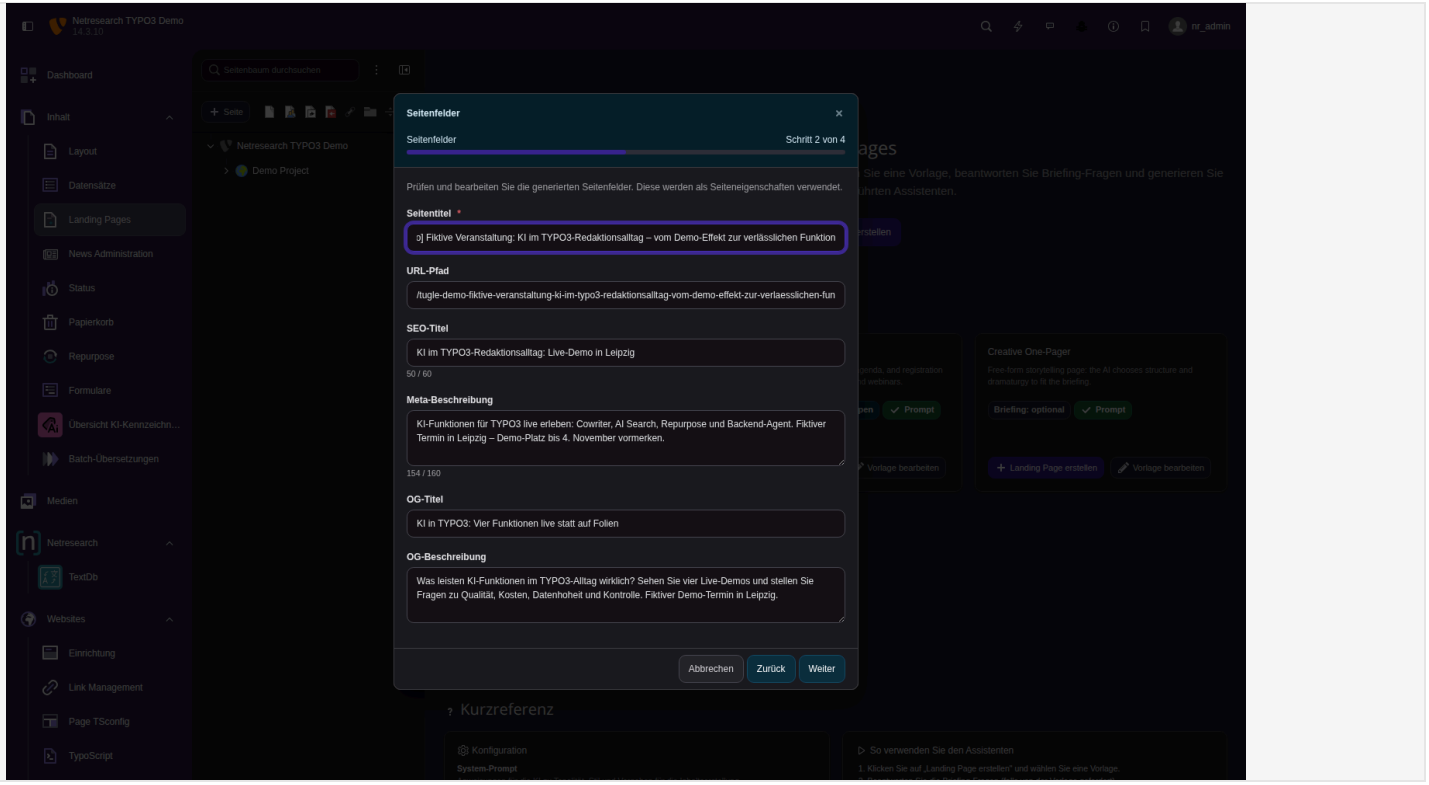
Passende, gefundene Inhalte der Website

Repurpose

Die Ausgangsquelle für neue Formate

Nur passende und benötigte Informationen als Kontext hinzufügen.

Ein plausibler Entwurf kann falsch sein



Prüfbeispiel: erfundene Frist in den Metadaten · 2.11.2026

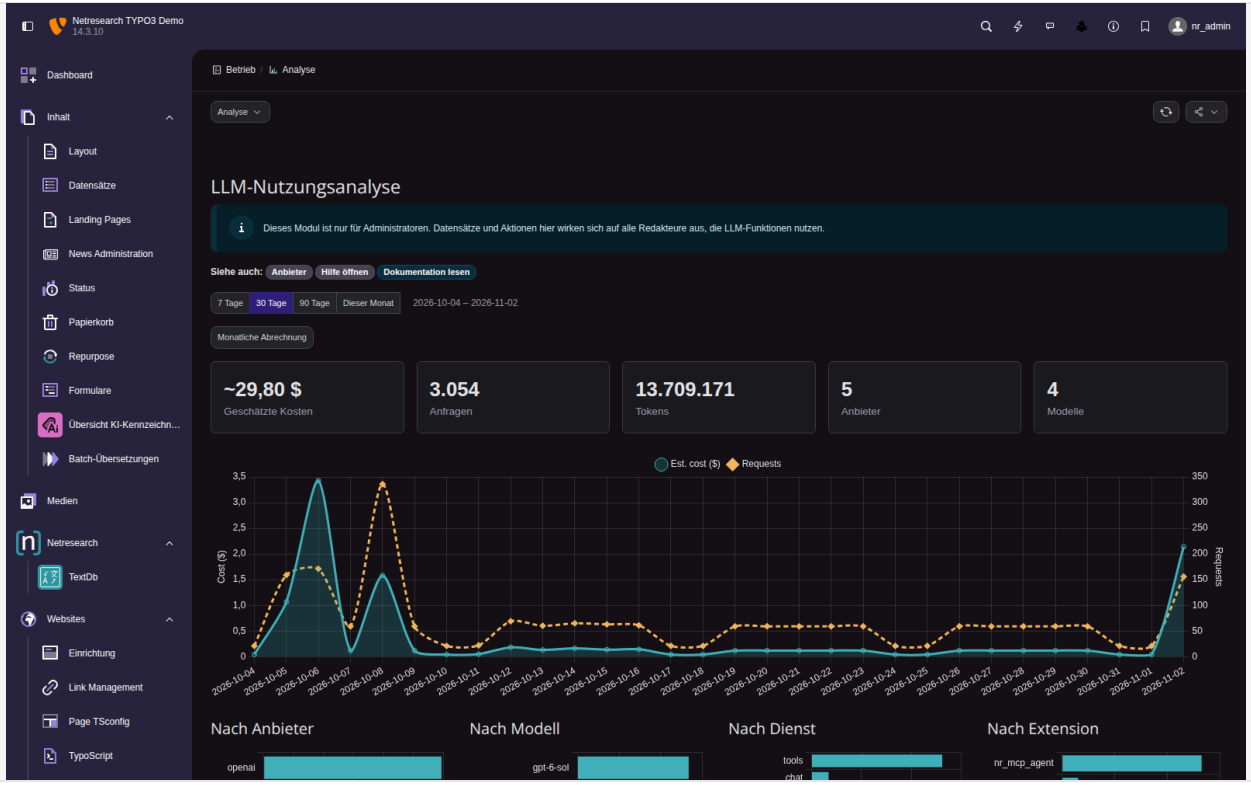
01 Im Briefing: keine Frist vorgegeben

02 Im Ergebnis: „bis 4. November“

03 Fachlich prüfen, bevor ihr freigebt

Landingpage-Vorschau

Kosten schätzen und Nutzung begrenzen



Historische Analyse der Demo-Instanz · 2.11.2026

01 Aufrufe und Tokens aufschlüsseln

02 Kosten anhand hinterlegter Preise schätzen

03 Budgets begrenzen weitere Aufrufe

Kostenübersicht öffnen

Welche Daten gehen an welchen Anbieter?

Empfänger

Welcher Anbieter verarbeitet die übertragenen Inhalte?

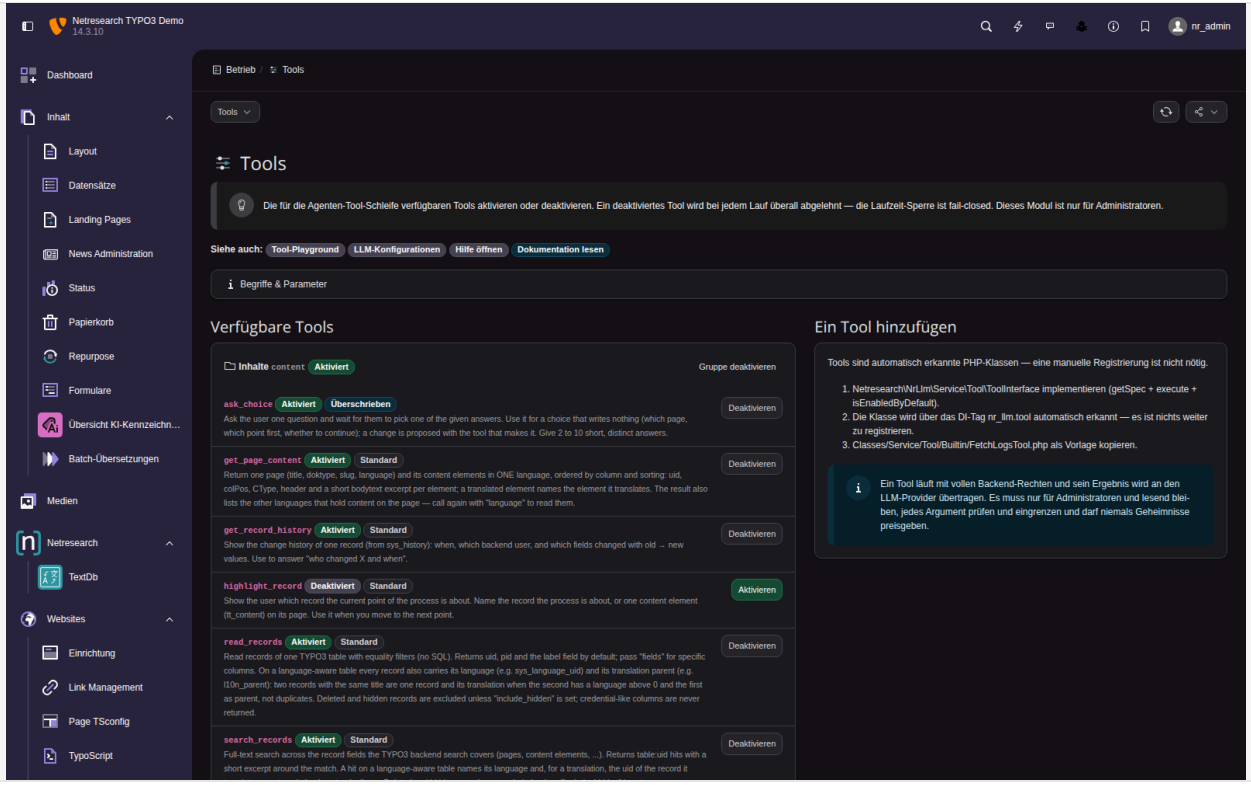
Zugang

Zugangsdaten zentral verwalten und Zugriffe steuern.

Datenumfang

Für jede Aufgabe die benötigten Inhalte gezielt auswählen.

Werkzeugzugriff und Schreibfreigaben



Werkzeugübersicht · Aufnahme vom 2.11.2026

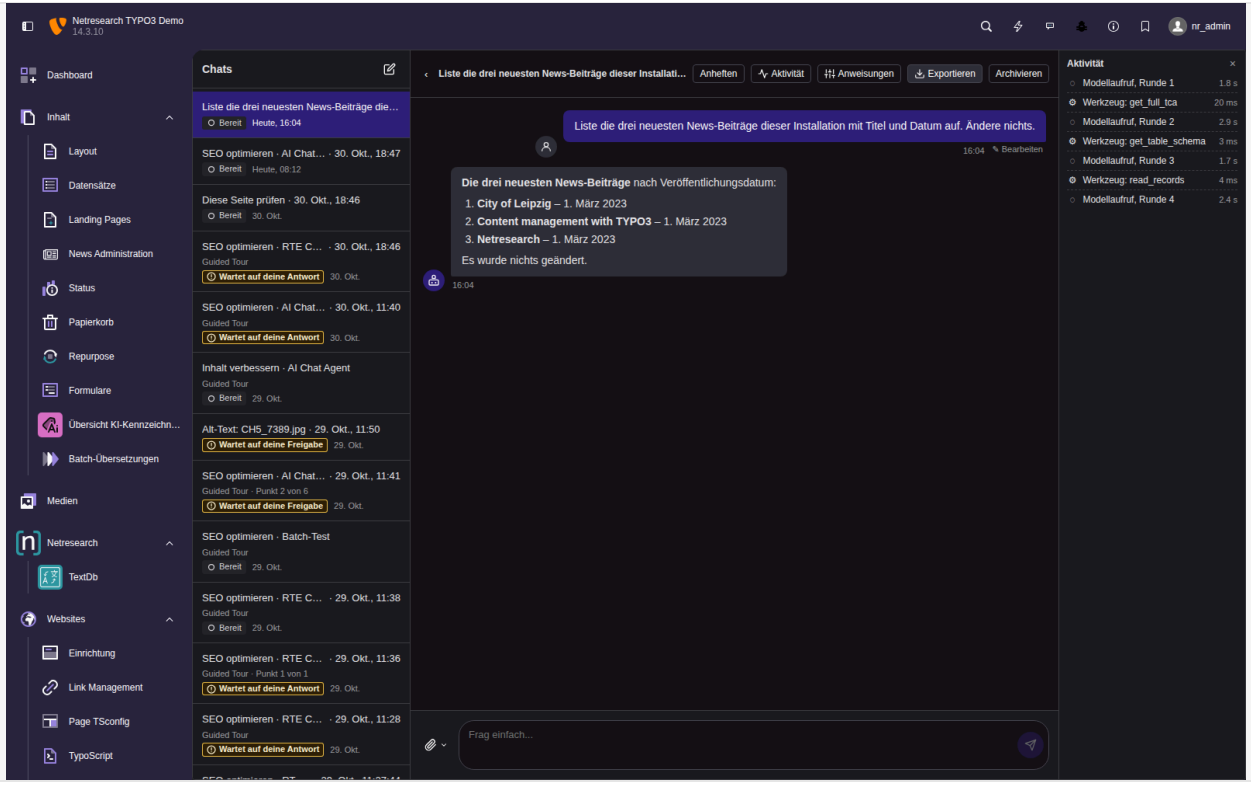
01 Lesende und schreibende Werkzeuge unterscheiden

02 Geplante Änderung vor Ausführung prüfen

03 Schreibende Standardwerkzeuge prüfen TYPO3-Rechte

Werkzeuge öffnen

Was hat der Agent tatsächlich getan?



Lesender Werkzeugverlauf · Aufnahme vom 2.11.2026

01 Werkzeug, Argumente und Ergebnis ansehen

02 Freigaben und ausgeführte Schritte nachvollziehen

03 Aus Protokollen gezielt Fehler untersuchen

KI-Chat öffnen

04

Agentenwege

Wie kann ein Agent mit TYPO3 arbeiten?

Im TYPO3-Backend

Auftrag im Backend

↓

Agent in TYPO3

↓

TYPO3-Werkzeuge

Aus einem externen Assistenten

Auftrag im eigenen Assistenten

↓

Externer Agent → TYPO3-MCP-Server

↓

TYPO3-Werkzeuge

01 MCP verbindet Agenten und Werkzeuge nach einem gemeinsamen Standard. Rechte und Freigaben bleiben nötig.

MCP-Architektur nachlesen | Beispiel: TYPO3 MCP Server

A2A: Ein Agent beauftragt einen anderen

01

Auftrag übergeben

Der Benutzer-Agent beauftragt einen TYPO3-Agenten.

→

02

Aufgabe bearbeiten

Der TYPO3-Agent arbeitet im eigenen Zuständigkeitsbereich.

→

03

Ergebnis zurückgeben

Der beauftragende Agent erhält eine Rückmeldung.

Ausblick: Identität, Rechte und Rückmeldung müssen zwischen den Agenten geklärt sein.

A2A und MCP: Einordnung lesen

05

Einordnung & Austausch

KI-Schichten und MCP-Server für TYPO3

KI-Schicht für Extensions

Gemeinsame Modellanbindung und Konfiguration für eigene Anwendungen.

Netresearch · nr-llm ↗

b13 · AiM (Alpha) ↗

NITSAN / T3Planet · AI Foundation ↗

MCP-Zugang zu TYPO3

TYPO3-Werkzeuge für externe Assistenten und deren Agentenabläufe.

hauptsache.net · TYPO3 MCP Server ↗

Marek Skopal · TYPO3 MCP Server ↗

AutoDudes · AI Suite MCP (Zusatz) ↗

KI-Schicht und MCP-Server lösen unterschiedliche Aufgaben und können sich ergänzen.

Eine Aufgabe wählen und ihren Nutzen prüfen

01

Aufgabe wählen

Einen konkreten Arbeitsablauf für den Einstieg auswählen.

→

02

Maßstab festlegen

Beispielinputs und passende Qualitätskriterien vorbereiten.

→

03

Einsatz klären

Datenweg, Kosten und Freigaben prüfen.

→

04

Nutzen bewerten

Ergebnisse und Nacharbeit bewerten, dann gezielt erweitern.

Welche Fragen sind noch offen?

01 Welche unserer Fragen sind beantwortet?

02 Wo bleiben Zweifel oder konkrete Hürden?

03 Welche Aufgabe würdet ihr als Erstes ausprobieren?

Links zu Demos und Anwendungen

Demo und Ausgangsmaterial

Öffentliche TYPO3-Demo ↗

TYPO3-Backend · Anmeldung nötig ↗

AI Search · Backend-Vorschau ↗

Landingpage-Entwurf · Backend-Vorschau ↗

Ausgangsartikel des Repurpose-Beispiels ↗

Anwendungen und Beschreibungen

Cowriter ↗

AI Search · Demo-Übersicht ↗

Repurpose ↗

Backend-Agent ↗

Landingpage ↗

Quellen zu KI-Grundlagen und MCP

Technische Grundlage

Modularer KI-Stack · Module und Voraussetzungen ↗

nr-llm · Projekt und Dokumentation ↗

nr-llm · für die Folien geprüfter Quellstand ↗

MCP · Architektur von Client und Server ↗

A2A und MCP · offizielle Einordnung ↗

Symfony AI Platform · PHP-Unterbau ↗

KI-Schichten und MCP-Server

b13 AiM · TYPO3-KI-Schicht ↗

NITSAN / T3Planet · AI Foundation ↗

hauptsache.net · TYPO3 MCP Server ↗

Marek Skopal · TYPO3 MCP Server ↗

AutoDudes · AI Suite MCP (Zusatz) ↗

Rückfragen: die passende Erklärung öffnen

Entwicklung

Formale Vorgaben prüfen 31

Skills einbinden 33

Agentenablauf verstehen 35

Eigene Extension anbinden 36

Kontext und Wissen liefern 37

Fehler und Kontextgrenzen 38

Such- und RAG-Grundlagen 47

Integration

MCP-Werkzeuge anbinden 34

Anbieter und Modell wechseln 39

Werkzeuge und Rechte begrenzen 40

Protokolle und Löschfristen 41

KI-Schichten vergleichen 45

TYPO3-MCP-Server vergleichen 46

Entscheidung

Qualität an Beispielen testen 32

Nutzen und Aufwand abwägen 42

Kosten und Budgets begrenzen 43

Produktivstart vorbereiten 44

Formale Vorgaben werden maschinell geprüft

01

Schema festlegen

Pflichtfelder, Datentypen, erlaubte Werte und Längen definieren.

→

02

Antwort erzeugen

Die Anwendung fordert eine strukturierte JSON-Ausgabe an.

→

03

Lokal prüfen

nr-llm prüft die Antwort gegen das unterstützte Schema.

→

04

Nur bei Fehlern

Ein Reparaturversuch; danach gültiges Ergebnis oder Fehler.

Beispiel: Ein Titel muss vorhanden sein und darf höchstens 60 Zeichen haben.

Ein gültiges Format belegt noch keine korrekte Aussage.

Funktionsweise und Quellbeleg

Qualität wird an Beispielaufgaben gemessen

01

Testfälle festlegen

Typische Aufgaben, erwartete Aussagen und Referenzantworten sammeln.

→

02

Antworten bewerten

Feste Regeln prüfen; optional bewertet ein weiteres Modell.

→

03

Läufe vergleichen

Bestehensquote und Bewertungen machen Verschlechterungen sichtbar.

Beispiel: Enthält die Antwort die belegten Öffnungszeiten und das erwartete Format?

Ein eigener Testlauf prüft Beispiele. Die Redaktion prüft den konkreten Entwurf.

Funktionsweise und Quellbeleg

Skills werden als Prompt-Kontext eingebunden

01

Quelle laden

SKILL.md aus GitHub synchronisieren; den Commit festhalten.

→

02

Inhalt freigeben

Administratoren prüfen und aktivieren die gewünschten Skills.

→

03

Aufgabe zuordnen

Skills an Konfiguration und gegebenenfalls Aufgabe anhängen.

→

04

Text einfügen

Geprüften Skill-Text vor dem Modellaufruf in den User-Prompt einfügen.

Beispiel: Gemeinsame Sprachregeln in der Konfiguration, Zusatzregeln an der Aufgabe.

Skill-Text liefert Kontext. Zugehörige Skripte und Dateien werden nicht ausgeführt oder nachgeladen.

Funktionsweise und Quellbeleg

MCP verbindet den Agenten mit Werkzeugen

01

Modell schlägt vor

Wählt ein angebotenes Werkzeug und formuliert seine Argumente.

→

02

nr-llm prüft

Nur Backend-Administratoren; zusätzliche Rechte und Freigaben begrenzen den Aufruf.

→

03

MCP-Client ruft auf

Sendet den Werkzeugaufruf per HTTP an den externen Server.

→

04

Ergebnis fließt zurück

Textresultat oder Fehler wird Kontext für den nächsten Modellschritt.

Vorher: Server konfigurieren, Werkzeugkatalog importieren und einzelne Werkzeuge aktivieren.

In diesem Ablauf ist TYPO3 der MCP-Client. Der externe Server führt das Werkzeug aus.

Funktionsweise und Quellbeleg

Ein Agent arbeitet in einer begrenzten Schleife

01

Auftrag

Ziel, Konfiguration, Skills und erlaubte Werkzeuge zusammenstellen.

→

02

Modell

Antwortet direkt oder schlägt einen Werkzeugaufruf vor.

→

03

Freigabe

Bei freigabepflichtigen Aufrufen pausieren und Entscheidung abwarten.

→

04

Werkzeug

Zulässigen Aufruf ausführen und Ergebnis zum Verlauf ergänzen.

→

05

Nächste Runde

Modell erhält das Ergebnis und bestimmt den nächsten Schritt.

↶ Das Werkzeugergebnis geht zurück an die Modellentscheidung – bis zur Antwort oder einer Grenze.

Das Modell schlägt Schritte vor; nr-llm steuert Ausführung, Pausen und Grenzen.

Funktionsweise und Quellbeleg

1 Möglichkeiten | 2 Fragen | 3 Antworten | 4 Agenten | 5 Einordnung

Wie bindet eine eigene Extension nr-llm ein?

01

Service injizieren

Passendes nr-llm-Interface über TYPO3 Dependency Injection verwenden.

→

02

Aufruf vorbereiten

Eingaben, Berechtigungen und die gewünschte Konfiguration bestimmen.

→

03

Ergebnis anfordern

Direkter Feature-Service oder wiederkehrender Task mit Prompt-Vorlage.

→

04

Ergebnis verarbeiten

Fehler behandeln, Ausgabe prüfen und fachliche Freigabe umsetzen.

Beispiel: completeForConfiguration($prompt, $configuration) liefert eine CompletionResponse.

nr-llm übernimmt die Modellanbindung. Die Extension gestaltet den fachlichen Ablauf.

Schnittstellen und Quellbeleg

Woher kommen Kontext und Wissen?

Mitgegeben

Prompt, aktuelle Inhalte und Skilltexte kommen aus der Anwendung.

Nachgeschlagen

Suchwerkzeuge liefern Auszüge mit Quellen aus vorhandenen TYPO3-Suchen.

Separat gepflegt

Indexaktualisierung und dauerhafte Vektorsuche bleiben eigene Integrationsaufgaben.

Das Modell sieht bereitgestellte oder abgerufene Quellen – keinen automatischen Wissensbestand.

Schnittstellen und Quellbeleg

Was passiert bei Fehlern oder zu viel Kontext?

Verbindung gestört

Begrenzt erneut versuchen; je nach Fehler eine konfigurierte Ersatzverbindung nutzen.

Zugriff falsch

Authentifizierung und Konfiguration korrigieren; Fehler an die Anwendung melden.

Kontext zu groß

Alte Chat-Runden können entfallen. Einen Einzelprompt muss die Anwendung passend aufteilen.

Die Anwendung braucht einen Fehlerpfad; Wiederholung und Fallback lösen nicht jedes Problem.

Schnittstellen und Quellbeleg

Ein Modellwechsel braucht einen Eignungstest

Anbindung wechseln

Die Anwendung nutzt eine benannte Konfiguration. Dahinter lassen sich Modell und Anbieter ändern.

Eignung prüfen

Benötigte Fähigkeiten, Kontextgröße und Parameter prüfen; typische Aufgaben erneut testen.

Rückfall erproben

Ersatzkonfigurationen bei Ausfällen testen. Datenziel, Qualität und Kosten jedes Ersatzes prüfen.

Eine gemeinsame Schnittstelle macht Modelle nicht fachlich austauschbar.

Modellwahl und Fähigkeiten | Rückfall im Quellcode

1 Möglichkeiten | 2 Fragen | 3 Antworten | 4 Agenten | 5 Einordnung

Werkzeugzugriff braucht mehrere Freigaben

01

Werkzeug aktivieren

Gruppe und einzelnes Werkzeug zentral freischalten.

→

02

Aufgabe begrenzen

Werkzeuggruppen je Konfiguration und Lauf einschränken.

→

03

Nutzerrechte prüfen

Adminpflicht und konkrete TYPO3-Rechte des ausführenden Nutzers beachten.

→

04

Änderung bestätigen

Bei den eingebauten Schreibwerkzeugen die geplante Änderung vorab freigeben.

Prüfbeispiel: Redaktion darf einen Seitentitel ändern, aber keine Systemdaten auslesen.

Eine Zustimmung ersetzt weder TYPO3-Rechte noch die erlaubte Werkzeugauswahl.

Werkzeugrechte im Quellcode | Schreibrechte und Freigaben

1 Möglichkeiten | 2 Fragen | 3 Antworten | 4 Agenten | 5 Einordnung

Betrieb braucht Messwerte und Löschroutinen

Nutzung auswerten

Anfragen, Tokens und geschätzte Kosten nach Anbieter, Modell und Nutzer ansehen.

Läufe nachvollziehen

Status, Schritte, Modellaufrufe und Entscheidungen in der Run-Timeline prüfen.

Aufbewahrung betreiben

Speicherumfang und Fristen wählen. Den Löschlauf regelmäßig ausführen lassen.

Kosten sind Schätzungen. Eine Löschfrist wirkt erst durch einen geplanten Löschlauf.

Run-Timeline und Grenzen | Aufbewahrung und Löschlauf

Wann lohnt sich eine gemeinsame KI-Grundlage?

Wiederkehrende Arbeit

Eine konkrete Aufgabe fällt regelmäßig an. Inhalte und Arbeitsablauf liegen bereits in TYPO3.

Gemeinsame Steuerung

Mehrere Anwendungen brauchen dieselben Anbieter, Modelle, Budgets oder Freigaberegeln.

Messbarer Nutzen

Im Pilot Zeit inklusive Nacharbeit, Ergebnisqualität und laufende Kosten mit dem bisherigen Ablauf vergleichen.

Einführen, wenn der gemessene Nutzen Integration und Betrieb rechtfertigt.

Module und Voraussetzungen vergleichen

Wie begrenzen wir Kosten?

Vor dem Aufruf

nr-llm prüft konfigurierte Nutzer- und Konfigurationslimits. Übergebene Plankosten fließen in die Prüfung ein.

Nach dem Aufruf

nr-llm erfasst die gemeldete Nutzung. Geschätzte Kosten stammen aus der Antwort oder den Modellpreisen.

Im Betrieb ergänzen

Preise und Anbieterrechnung abgleichen. Für eine harte Kostengrenze zusätzlich eine Reservierung oder Sperre einplanen.

Parallele Aufrufe können das Budget überschreiten. Die Vorabprüfung garantiert keine harte Kostengrenze.

Budgets und ihre Grenzen

Was muss vor dem produktiven Einsatz geklärt sein?

Datenfluss

Welche Daten gehen wohin? Anbieter, Region und Aufbewahrung für Modell, Ersatzpfad und Werkzeuge festlegen.

Rechte und Freigabe

Wer darf lesen, schreiben und veröffentlichen? Zuständige Person und Freigabeschritt je Ablauf benennen.

Qualität

Typische Aufgaben und kritische Fehler testen. Vorab festlegen, welche Ergebnisse Menschen prüfen müssen.

Betrieb

Verantwortung für Kosten, Protokolle und Updates zuweisen. Abschaltung und Ablauf bei Ausfall festlegen.

Produktiv starten, wenn Datenwege, Freigaben, Qualitätsmaßstab und Betriebsverantwortung feststehen.

Datenflüsse und Verantwortung im Überblick

Welche gemeinsame KI-Schicht passt zur Integration?

Netresearch nr-llm

Gemeinsame KI-Dienste, Konfigurationen, Tasks und Werkzeugsteuerung für integrierte Anwendungen.

b13 AiM

TYPO3-Proxy mit Routing und Regeln; Provider-Anbindung über Symfony-AI-Bridges. Als Alpha gekennzeichnet.

T3Planet AI Foundation

Gemeinsame Service-API, Provider, Prompts, Kontext und Nutzungsdaten; ergänzt um MCP-Zugriff.

An der eigenen Extension prüfen: API, Aufgabensteuerung, Rechte, Datenweg und Betriebsanforderungen.

b13 AiM: Konzept und Schnittstellen

AI Foundation: Extension-Integration

MCP-Server: welcher Zugang zu TYPO3?

hauptsache.net

Eigenständiger Server. Datensätze über Workspaces bearbeiten; eigene PHP-Werkzeuge ergänzen.

Marek Skopal

Eigenständiger Server. Direkte Verwaltung als Hauptmodus; Workspaces optional und TCA-Werkzeuge erweiterbar.

AutoDudes MCP

Zusatz zur AI Suite. TYPO3-Inhalte und ausgewählte KI-Funktionen über die Dienste und Rechte der Suite anbieten.

MCP gibt Assistenten Werkzeugzugriff. KI-Anbindung, Rechte und Freigaben bleiben eigene Entscheidungen.

hauptsache.net · MCP-Server | Marek Skopal · MCP-Server

Welche Grundlagen gibt es für eigene Wissenssuche?

EXT:index

Indexierungsframework für TYPO3-Inhalte: Aufbereitung und Weitergabe über Ereignisse oder Webhooks.

SEAL + seal_ai

Suchabstraktion, Embeddings und Vektorspeicher; optional lesender MCP-Suchzugang. seal_ai: Proof of Concept.

smart_search

Dienste für Vektoren, semantische Suche und RAG in eigenen Extensions. Als Alpha gekennzeichnet.

Diese Bausteine kümmern sich um Wissenszugriff. Eine KI-Schicht allein pflegt noch keinen Suchindex.

Indexierungsframework | SEAL / Symfony AI | RAG-Bausteine